

The structures of Riktig

Every class in theories/Structure/ – carriers, operations with their types, and laws – as Coq states them, generated by scripts/structures-md.py

Magma (M)

```
ops: add (+) : M → M → M
• equivalence: Equivalence equ
• op_proper: Proper (equ ==> equ ==> equ) add
```

Semigroup (S) extends Magma

```
ops: add (+) : S → S → S
• is_associative: ∀ a b x : S, a + b + x == a + (b + x)
```

Monoid (M) extends Semigroup

```
ops: zero (0) : M, add (+) : M → M → M
• is_left_identity: ∀ x : M, 0 + x == x
• is_right_identity: ∀ x : M, x + 0 == x
```

Group (G) extends Monoid

```
ops: zero (0) : G, add (+) : G → G → G, neg (- x) : G → G
• neg_proper: Proper (equ ==> equ) neg
• is_left_inverse: ∀ a : G, - a + a == 0
• is_right_inverse: ∀ a : G, a - a == 0
```

AbGroup (A) extends Group

```
ops: zero (0) : A, add (+) : A → A → A, neg (- x) : A → A
• is_commutative: ∀ a b : A, a + b == b + a
```

Semiring (R) extends Monoid (add_monoid), Monoid (mul_monoid)

```
ops: zero (0) : R, add (+) : R → R → R, one (1) : R, mult (*) : R → R → R
• add_commutative: ∀ a b : R, a + b == b + a
• is_left_distributive: ∀ a x y : R, a * (x + y) == a * x + a * y
• is_right_distributive: ∀ x y b : R, (x + y) * b == x * b + y * b
• mul_zero_left: ∀ x : R, 0 * x == 0
• mul_zero_right: ∀ x : R, x * 0 == 0
```

CommSemiring (R) extends Semiring

```
ops: zero (0) : R, add (+) : R → R → R, one (1) : R, mult (*) : R → R → R
• mul_commutative: ∀ a b : R, a * b == b * a
```

Peano (N)

```
ops: zero (0) : N, succ : N → N
• equivalence: Equivalence equ
• succ_proper: Proper (equ ==> equ) succ
• succ_injective: ∀ a b : N, succ a == succ b → a == b
• succ_not_zero: ∀ n : N, succ n /= 0
• nat_induction: ∀ P : N → Prop, Proper (equ ==> iff) P → P 0 → (∀ n : N, P n → P (succ n)) → ∀ n : N, P n
```

PeanoArith (N) extends Peano

```
ops: zero (0) : N, succ : N → N, one (1) : N, add (+) : N → N → N, mult (*) : N → N → N
• add_proper: Proper (equ ==> equ ==> equ) add
• mul_proper: Proper (equ ==> equ ==> equ) mult
• one_succ: (1 : N) == succ 0
• add_zero: ∀ a : N, a + 0 == a
• add_succ: ∀ a b : N, a + succ b == succ (a + b)
• mul_zero: ∀ a : N, a * 0 == 0
• mul_succ: ∀ a b : N, a * succ b == a * b + a
```

Naturals (N) extends Peano, CommSemiring

```
ops: zero (0) : N, succ : N → N, one (1) : N, add (+) : N → N → N, mult (*) : N → N → N
• succ_add: ∀ n : N, succ n == n + 1
```

NatOrder (N) over Naturals N

```
ops: le (<=) : N → N → Prop
• le_proper: Proper (equ ==> equ ==> iff) le
• le_add: ∀ a b : N, a <= b ↔ exists c : N, a + c == b
```

Digits (D B N) over NatOrder N

```
ops: val : HasRadix D N, valD : D → N, valB : B → N, add_digit : D → D → B → D, carry : D → D → B → B
• equivalence_digit: Equivalence equ
• equivalence_carry: Equivalence equ
• val_digit_proper: Proper (equ ==> equ) (val : D → N)
• val_carry_proper: Proper (equ ==> equ) (val : B → N)
• add_digit_proper: Proper (equ ==> equ ==> equ ==> equ) add_digit
• carry_proper: Proper (equ ==> equ ==> equ ==> equ) carry
```

- `val_digit_lt`: $\forall d : D, \text{val } d < @\text{radix } D \text{ N val}$
- `val_carry_lt`: $\forall b : B, \text{val } b < 1 + 1$
- `add_digit_spec`: $\forall (x \ y : D) (c : B), \text{val } (\text{add_digit } x \ y \ c) + @\text{radix } D \text{ N val} * \text{val } (\text{carry } x \ y \ c) == \text{val } x + \text{val } y + \text{val } c$

NatRec (N) over Peano N

ops: `natrec` : $\forall A : \text{Type}, A \rightarrow (N \rightarrow A \rightarrow A) \rightarrow N \rightarrow A$

- `natrec_proper`: $\forall (A : \text{Type}) (eA : \text{EqRel } A) (EA : \text{Equivalence } eA) (a : A) (f : N \rightarrow A \rightarrow A), \text{Proper } (\text{equ} ==> \text{equ}) f \rightarrow \text{Proper } (\text{equ} ==> \text{equ}) (\text{natrec } A \ a \ f)$
- `natrec_zero`: $\forall (A : \text{Type}) (eA : \text{EqRel } A) (EA : \text{Equivalence } eA) (a : A) (f : N \rightarrow A \rightarrow A), \text{natrec } A \ a \ f \ 0 == a$
- `natrec_succ`: $\forall (A : \text{Type}) (eA : \text{EqRel } A) (EA : \text{Equivalence } eA) (a : A) (f : N \rightarrow A \rightarrow A) (n : N), \text{natrec } A \ a \ f \ (\text{succ } n) == f \ n \ (\text{natrec } A \ a \ f \ n)$

BinaryNumber (N)

ops: `zero` (`0`) : $N, \text{bit0} : N \rightarrow N, \text{bit1} : N \rightarrow N$

- `equivalence`: $\text{Equivalence } \text{equ}$
- `bit0_proper`: $\text{Proper } (\text{equ} ==> \text{equ}) \text{ bit0}$
- `bit1_proper`: $\text{Proper } (\text{equ} ==> \text{equ}) \text{ bit1}$
- `bit0_injective`: $\forall a \ b : N, \text{bit0 } a == \text{bit0 } b \rightarrow a == b$
- `bit1_injective`: $\forall a \ b : N, \text{bit1 } a == \text{bit1 } b \rightarrow a == b$
- `bit0_zero`: $\text{bit0 } 0 == 0$
- `bit1_not_zero`: $\forall n : N, \text{bit1 } n \neq 0$
- `bit0_ne_bit1`: $\forall n \ m : N, \text{bit0 } n \neq \text{bit1 } m$
- `bin_induction`: $\forall P : N \rightarrow \text{Prop}, \text{Proper } (\text{equ} ==> \text{iff}) \ P \rightarrow P \ 0 \rightarrow (\forall n : N, P \ n \rightarrow P \ (\text{bit0 } n)) \rightarrow (\forall n : N, P \ n \rightarrow P \ (\text{bit1 } n)) \rightarrow \forall n : N, P \ n$

BinRec (N) over BinaryNumber N

ops: `binrec` : $\forall A : \text{Type}, A \rightarrow (N \rightarrow A \rightarrow A) \rightarrow (N \rightarrow A \rightarrow A) \rightarrow N \rightarrow A$

- `binrec_proper`: $\forall (A : \text{Type}) (eA : \text{EqRel } A) (EA : \text{Equivalence } eA) (a : A) (f0 \ f1 : N \rightarrow A \rightarrow A), \text{Proper } (\text{equ} ==> \text{equ}) f0 \rightarrow \text{Proper } (\text{equ} ==> \text{equ}) f1 \rightarrow \text{Proper } (\text{equ} ==> \text{equ}) (\text{binrec } A \ a \ f0 \ f1)$
- `binrec_ext`: $\forall (A : \text{Type}) (eA : \text{EqRel } A) (EA : \text{Equivalence } eA) (a \ a' : A) (f0 \ f0' \ f1 \ f1' : N \rightarrow A \rightarrow A), (\forall k : N, \text{Proper } (\text{equ} ==> \text{equ}) (f0 \ k)) \rightarrow (\forall k : N, \text{Proper } (\text{equ} ==> \text{equ}) (f1 \ k)) \rightarrow a == a' \rightarrow (\forall (k : N) (x : A), f0 \ k \ x == f0' \ k \ x) \rightarrow (\forall (k : N) (x : A), f1 \ k \ x == f1' \ k \ x) \rightarrow \forall n : N, \text{binrec } A \ a \ f0 \ f1 \ n == \text{binrec } A \ a' \ f0' \ f1' \ n$
- `binrec_zero`: $\forall (A : \text{Type}) (eA : \text{EqRel } A) (EA : \text{Equivalence } eA) (a : A) (f0 \ f1 : N \rightarrow A \rightarrow A), \text{binrec } A \ a \ f0 \ f1 \ 0 == a$
- `binrec_bit0`: $\forall (A : \text{Type}) (eA : \text{EqRel } A) (EA : \text{Equivalence } eA) (a : A) (f0 \ f1 : N \rightarrow A \rightarrow A) (n : N), n \neq 0 \rightarrow \text{binrec } A \ a \ f0 \ f1 \ (\text{bit0 } n) == f0 \ n \ (\text{binrec } A \ a \ f0 \ f1 \ n)$
- `binrec_bit1`: $\forall (A : \text{Type}) (eA : \text{EqRel } A) (EA : \text{Equivalence } eA) (a : A) (f0 \ f1 : N \rightarrow A \rightarrow A) (n : N), \text{binrec } A \ a \ f0 \ f1 \ (\text{bit1 } n) == f1 \ n \ (\text{binrec } A \ a \ f0 \ f1 \ n)$

BinaryArith (N) extends BinaryNumber

ops: `zero` (`0`) : $N, \text{bit0} : N \rightarrow N, \text{bit1} : N \rightarrow N, \text{succ} : N \rightarrow N, \text{one} (`1`) : $N, \text{add} (`+`) : $N \rightarrow N \rightarrow N, \text{mult} (`*`) : $N \rightarrow N \rightarrow N$$$$

- `succ_proper`: $\text{Proper } (\text{equ} ==> \text{equ}) \text{ succ}$
- `add_proper`: $\text{Proper } (\text{equ} ==> \text{equ}) \text{ add}$
- `mul_proper`: $\text{Proper } (\text{equ} ==> \text{equ}) \text{ mult}$
- `one_bit1`: $(1 : N) == \text{bit1 } 0$
- `succ_zero`: $\text{succ } 0 == \text{bit1 } 0$
- `succ_bit0`: $\forall n : N, n \neq 0 \rightarrow \text{succ } (\text{bit0 } n) == \text{bit1 } n$
- `succ_bit1`: $\forall n : N, \text{succ } (\text{bit1 } n) == \text{bit0 } (\text{succ } n)$
- `add_zero_l`: $\forall y : N, 0 + y == y$
- `add_bit0_zero`: $\forall n : N, n \neq 0 \rightarrow \text{bit0 } n + 0 == \text{bit0 } n$
- `add_bit0_bit0`: $\forall n \ m : N, n \neq 0 \rightarrow m \neq 0 \rightarrow \text{bit0 } n + \text{bit0 } m == \text{bit0 } (n + m)$
- `add_bit0_bit1`: $\forall n \ m : N, n \neq 0 \rightarrow \text{bit0 } n + \text{bit1 } m == \text{bit1 } (n + m)$
- `add_bit1_zero`: $\forall n : N, \text{bit1 } n + 0 == \text{bit1 } n$
- `add_bit1_bit0`: $\forall n \ m : N, m \neq 0 \rightarrow \text{bit1 } n + \text{bit0 } m == \text{bit1 } (n + m)$
- `add_bit1_bit1`: $\forall n \ m : N, \text{bit1 } n + \text{bit1 } m == \text{bit0 } (\text{succ } (n + m))$
- `mul_zero_l`: $\forall y : N, 0 * y == 0$
- `mul_bit0`: $\forall n \ y : N, n \neq 0 \rightarrow \text{bit0 } n * y == \text{bit0 } (n * y)$
- `mul_bit1`: $\forall n \ y : N, \text{bit1 } n * y == \text{bit0 } (n * y) + y$

Finite (IN) over NatOrder N

ops: `card` : $N, \text{index} : I \rightarrow N, \text{elem} : N \rightarrow I$

- `equivalence`: $\text{Equivalence } \text{equ}$
- `index_proper`: $\text{Proper } (\text{equ} ==> \text{equ}) \text{ index}$
- `elem_proper`: $\text{Proper } (\text{equ} ==> \text{equ}) \text{ elem}$
- `index_lt`: $\forall i : I, \text{index } i < \text{card}$
- `elem_index`: $\forall i : I, \text{elem } (\text{index } i) == i$
- `index_elem`: $\forall k : N, k < \text{card} \rightarrow \text{index } (\text{elem } k) == k$

Table (I A T)

ops: `get` : $T \rightarrow I \rightarrow A, \text{tabulate} : (I \rightarrow A) \rightarrow T$

- `equivalence`: $\text{Equivalence } \text{equ}$
- `get_proper`: $\text{Proper } (\text{equ} ==> \text{equ}) \text{ get}$
- `tabulate_proper`: $\text{Proper } (\text{pointwise_relation } I \ \text{equ} ==> \text{equ}) \text{ tabulate}$

- `get_tabulate`: $\forall (f : I \rightarrow A) (i : I), \text{get } (\text{tabulate } f) i == f i$
- `tabulate_get`: $\forall t : T, \text{tabulate } (\text{get } t) == t$

Equiv (A B)

ops: `to` : $A \rightarrow B$, `from` : $B \rightarrow A$

- `to_proper`: `Proper (equ ==> equ) to`
- `from_proper`: `Proper (equ ==> equ) from`
- `from_to`: $\forall a : A, \text{from } (\text{to } a) == a$
- `to_from`: $\forall b : B, \text{to } (\text{from } b) == b$

EquDec (A)

- `equ_dec`: $\forall a b : A, \text{Decidable } (a == b)$

LtDec (A)

- `lt_dec`: $\forall a b : A, \text{Decidable } (a < b)$

LeDec (A)

- `le_dec`: $\forall a b : A, \text{Decidable } (a \leq b)$

BigOp (I A T) over `Monoid A`, `Table I A T` get `tabulate`

ops: `bigop` : $T \rightarrow A$

- `bigop_proper`: `Proper (equ ==> equ) bigop`
- `bigop_zero`: `bigop (tabulate ( $\lambda \_ \Rightarrow 0$ )) == 0`
- `bigop_add`: $\forall s t : T, \text{bigop } (\text{tabulate } (\lambda i \Rightarrow \text{get } s i + \text{get } t i)) == \text{bigop } s + \text{bigop } t$
- `bigop_single`: $\forall (t : T) (i : I) (a : A), \text{get } t i == a \rightarrow (\forall j : I, j \neq i \rightarrow \text{get } t j == 0) \rightarrow \text{bigop } t == a$

Ring (R) extends `AbGroup`, `Monoid`

ops: `zero` (`0`) : R , `add` (`+`) : $R \rightarrow R \rightarrow R$, `neg` (`- x`) : $R \rightarrow R$, `one` (`1`) : R , `mult` (`*`) : $R \rightarrow R \rightarrow R$

- `is_left_distributive`: $\forall a x y : R, a * (x + y) == a * x + a * y$
- `is_right_distributive`: $\forall x y b : R, (x + y) * b == x * b + y * b$

RingHom (A B)

ops: `h` : $A \rightarrow B$

- `hom_proper`: `Proper (equ ==> equ) h`
- `hom_additive`: $\forall x y : A, h (x + y) == h x + h y$
- `hom_multiplicative`: $\forall x y : A, h (x * y) == h x * h y$
- `hom_one`: `h 1 == (1 : B)`

Field (F) extends `Ring`

ops: `zero` (`0`) : F , `add` (`+`) : $F \rightarrow F \rightarrow F$, `neg` (`- x`) : $F \rightarrow F$, `one` (`1`) : F , `mult` (`*`) : $F \rightarrow F \rightarrow F$, `inv` : $F \rightarrow F$

- `inv_proper`: `Proper (equ ==> equ) inv`
- `is_mul_commutative`: $\forall a b : F, a * b == b * a$
- `is_mul_left_inverse`: $\forall a : F, \text{nonzero } F a \rightarrow \text{inv } a * a == 1$
- `nontrivial`: `(1 : F) != 0`

MinusOneNotASquare (F) over `Field F`

- `two_squares_vanish`: $\forall a b : F, a * a + b * b == 0 \rightarrow a == 0$

ScaleLaws (F V) over `Field F`

ops: `add` (`+`) : $V \rightarrow V \rightarrow V$, `scale` (`*`) : $F \rightarrow V \rightarrow V$

- `scale_proper`: `Proper (equ ==> equ ==> equ) scale`
- `is_left_distributive`: $\forall (a : F) (x y : V), a * (x + y) == a * x + a * y$
- `is_right_distributive`: $\forall (x y : F) (b : V), (x + y) * b == x * b + y * b$
- `is_associative`: $\forall (a b : F) (x : V), (a * b) * x == a * (b * x)$
- `is_left_identity`: $\forall x : V, 1 * x == x$

VectorSpace (F V) over `Field F` extends `AbGroup`, `ScaleLaws`

ops: `zero` (`0`) : V , `add` (`+`) : $V \rightarrow V \rightarrow V$, `neg` (`- x`) : $V \rightarrow V$, `scale` (`*`) : $F \rightarrow V \rightarrow V$

Linear (F U V)

ops: `f` : $U \rightarrow V$

- `proper`: `Proper (equ ==> equ) f`
- `is_additive`: $\forall x y : U, f (x + y) == f x + f y$
- `is_homogeneous`: $\forall (a : F) (x : U), f (a * x) == a * f x$

InnerProductSpace (F V) over `VectorSpace F V`

- `inner_proper`: `Proper (equ ==> equ ==> equ) (@inner F V _)`
- `inner_additive`: $\forall v : V, \text{Additive } V F \text{ add add } (\lambda u \Rightarrow \langle u, v \rangle)$
- `inner_homogeneous`: $\forall v : V, \text{Homogeneous } F V F \text{ mult } (\lambda a \Rightarrow a) (\lambda u \Rightarrow \langle u, v \rangle)$
- `is_commutative`: $\forall a b : V, \langle a, b \rangle == \langle b, a \rangle$

Integers (Z) extends `Ring`

ops: `zero` (`0`) : Z , `add` (`+`) : $Z \rightarrow Z \rightarrow Z$, `neg` (`- x`) : $Z \rightarrow Z$, `one` (`1`) : Z , `mult` (`*`) : $Z \rightarrow Z \rightarrow Z$

- `mul_commutative`: $\forall a b : Z, a * b == b * a$
- `int_induction`: $\forall P : Z \rightarrow \text{Prop}, \text{Proper } (\text{equ} ==> \text{iff}) P \rightarrow P 0 \rightarrow (\forall z : Z, P z \rightarrow P (z + 1)) \rightarrow (\forall z : Z, P z \rightarrow P (z + - (1 : Z))) \rightarrow \forall z : Z, P z$

Rationals (Q Z) over `Field Q`, `Integers Z` extends `RingHom`

ops: `cast` : $Z \rightarrow Q$

- `cast_injective`: $\forall a b : Z, \text{cast } a == \text{cast } b \rightarrow a == b$
- `is_fraction`: $\forall q : Q, \text{exists } a b : Z, \text{cast } b \neq 0 \wedge q * \text{cast } b == \text{cast } a$

OrderedField (F) over `Field F`

ops: `le` (`<=`) : $F \rightarrow F \rightarrow \text{Prop}$, `abs` : $F \rightarrow F$

- `le_proper`: `Proper (equ ==> equ ==> iff) le`
- `abs_proper`: `Proper (equ ==> equ) abs`

```

• le_reflexive: Reflexive le
• le_transitive: Transitive le
• le_antisymmetric: Antisymmetric F equ le
• le_total:  $\forall a b : F, a \leq b \vee b \leq a$ 
• le_stable:  $\forall a b : F, \sim \sim a \leq b \rightarrow a \leq b$ 
• add_le_mono_l:  $\forall a b c : F, a \leq b \rightarrow c + a \leq c + b$ 
• mul_nonneg:  $\forall a b : F, 0 \leq a \rightarrow 0 \leq b \rightarrow 0 \leq a * b$ 
• abs_of_nonneg:  $\forall a : F, 0 \leq a \rightarrow \text{abs } a = a$ 
• abs_neg:  $\forall a : F, \text{abs } (- a) = \text{abs } a$ 
Reals (R) over OrderedField R
ops: lt :  $\text{lt} : R \rightarrow R \rightarrow \text{Set}, \text{inv\_apart} : \forall x : R, \text{lt } x \ 0 + \text{lt } 0 \ x \rightarrow R, \text{abs} : R \rightarrow R, \text{approx} : R \rightarrow Q \rightarrow Q, \text{cast} : Q \rightarrow R$ 
• complete:  $\forall P : \text{Pred } R, (\text{exists } x : R, P \ x) \rightarrow (\text{exists } b : R, \forall x : R, P \ x \rightarrow x \leq b) \rightarrow \text{exists } s : R, (\forall x : R, P \ x \rightarrow x \leq s) \wedge (\forall b : R, (\forall x : R, P \ x \rightarrow x \leq b) \rightarrow s \leq b)$ 
ApproxSqrt (F) over OrderedField F
ops: approx_sqrt :  $F \rightarrow F \rightarrow F$ 
• approx_sqrt_nonneg:  $\forall x \text{ eps} : F, 0 \leq \text{approx\_sqrt } x \text{ eps}$ 
• approx_sqrt_le:  $\forall x \text{ eps} : F, 0 \leq x \rightarrow 0 < \text{eps} \rightarrow \text{approx\_sqrt } x \text{ eps} * \text{approx\_sqrt } x \text{ eps} \leq x$ 
• approx_sqrt_close:  $\forall x \text{ eps} : F, 0 \leq x \rightarrow 0 < \text{eps} \rightarrow x \leq (\text{approx\_sqrt } x \text{ eps} + \text{eps}) * (\text{approx\_sqrt } x \text{ eps} + \text{eps})$ 
ConstructiveReals (R Q N) over Ring R, Commutative R R mult, OrderedField Q, NatOrder N extends RingHom
ops: lt (<) :  $R \rightarrow R \rightarrow \text{Set}, \text{inv\_apart} : \forall x : R, \text{lt } x \ 0 + \text{lt } 0 \ x \rightarrow R, \text{abs} : R \rightarrow R, \text{approx} : R \rightarrow Q \rightarrow Q, \text{cast} : Q \rightarrow R$ 
• lt_asym:  $\forall x y : R, x < y \rightarrow y < x \rightarrow \text{False}$ 
• lt_trans:  $\forall x y z : R, x < y \rightarrow y < z \rightarrow x < z$ 
• lt_cotrans:  $\forall x y z : R, x < z \rightarrow ((x < y) + (y < z))\% \text{type}$ 
• lt_equ_l:  $\forall x x' y : R, x = x' \rightarrow x < y \rightarrow x' < y$ 
• lt_equ_r:  $\forall x y y' : R, y = y' \rightarrow x < y \rightarrow x < y'$ 
• equ_of_le:  $\forall x y : R, x \leq y \rightarrow y \leq x \rightarrow x = y$ 
• cast_lt:  $\forall q r : Q, q < r \rightarrow \text{cast } q < \text{cast } r$ 
• lt_cast:  $\forall q r : Q, \text{cast } q < \text{cast } r \rightarrow q < r$ 
• zero_lt_one:  $(0 : R) < 1$ 
• add_lt_mono_l:  $\forall x y z : R, y < z \rightarrow x + y < x + z$ 
• add_lt_reg_l:  $\forall x y z : R, x + y < x + z \rightarrow y < z$ 
• mul_pos:  $\forall x y : R, 0 < x \rightarrow 0 < y \rightarrow 0 < x * y$ 
• mul_le_mono_l:  $\forall c x y : R, 0 \leq c \rightarrow x \leq y \rightarrow c * x \leq c * y$ 
• inv_l:  $\forall (x : R) (Hx : ((x < 0) + (0 < x))\% \text{type}), \text{inv\_apart } x \ Hx * x = 1$ 
• inv_pos:  $\forall (x : R) (Hx : ((x < 0) + (0 < x))\% \text{type}), 0 < x \rightarrow 0 < \text{inv\_apart } x \ Hx$ 
• dense:  $\forall x y : R, x < y \rightarrow \{q : Q \mid ((x < \text{cast } q) * (\text{cast } q < y))\% \text{type}\}$ 
• archimedean:  $\forall x : R, \{q : Q \mid x < \text{cast } q\}$ 
• abs_proper: Proper (equ ==> equ) abs
• abs_le_iff:  $\forall x y : R, (x \leq y \wedge - x \leq y) \leftrightarrow \text{abs } x \leq y$ 
• abs_mul:  $\forall x y : R, \text{abs } (x * y) = \text{abs } x * \text{abs } y$ 
• approx_close:  $\forall (x : R) (\text{eps} : Q), 0 < \text{eps} \rightarrow \text{abs } (x + - \text{cast } (\text{approx } x \text{ eps})) \leq \text{cast } \text{eps}$ 
• complete:  $\forall u : N \rightarrow R, (\forall \text{eps} : Q, 0 < \text{eps} \rightarrow \{n : N \mid \forall i j : N, n \leq i \rightarrow n \leq j \rightarrow \text{abs } (u \ i + - u \ j) \leq \text{cast } \text{eps}\}) \rightarrow \{l : R \mid \forall \text{eps} : Q, 0 < \text{eps} \rightarrow \{n : N \mid \forall i : N, n \leq i \rightarrow \text{abs } (u \ i + - l) \leq \text{cast } \text{eps}\}\}$ 
Trig (R) over Ring R, Commutative R R mult
ops: sin :  $R \rightarrow R, \text{cos} : R \rightarrow R$ 
• sin_proper: Proper (equ ==> equ) sin
• cos_proper: Proper (equ ==> equ) cos
• sin_zero:  $\sin 0 = 0$ 
• cos_zero:  $\cos 0 = 1$ 
• sin_add:  $\forall a b : R, \sin (a + b) = \sin a * \cos b + \cos a * \sin b$ 
• cos_add:  $\forall a b : R, \cos (a + b) = \cos a * \cos b - (\sin a * \sin b)$ 
• sin_neg:  $\forall a : R, \sin (- a) = - \sin a$ 
• cos_neg:  $\forall a : R, \cos (- a) = \cos a$ 
Category (O)
ops: cat_id :  $\forall x : O, x \rightarrow x, \text{comp } (\circ) : \forall x y z : O, y \rightarrow x \rightarrow z \rightarrow y \rightarrow x \rightarrow z$ 
• hom_equivalence:  $\forall x y : O, \text{Equivalence } (@\text{equ } (x \rightarrow y) (\text{hom\_equ } x \ y))$ 
• comp_proper:  $\forall x y z : O, \text{Proper } (\text{equ} ==> \text{equ} ==> \text{equ}) (@\text{comp } 0 \ \text{hom} \ \text{comp } x \ y \ z)$ 
• is_left_identity:  $\forall x y : O, \text{LeftIdentity } (y \rightarrow y) (x \rightarrow y) (@\text{comp } 0 \ \text{hom} \ \text{comp } x \ y \ y) \ \text{cat\_id}$ 
• is_right_identity:  $\forall x y : O, \text{RightIdentity } (x \rightarrow x) (x \rightarrow y) (@\text{comp } 0 \ \text{hom} \ \text{comp } x \ x \ y) \ \text{cat\_id}$ 
• comp_assoc:  $\forall (w x y z : O) (f : y \rightarrow z) (g : x \rightarrow y) (h : w \rightarrow x), f \circ (g \circ h) = (f \circ g) \circ h$ 
Functor (C D) over Category C, Category D
ops: fmap :  $\forall x y : C, x \rightarrow y \rightarrow F \ x \rightarrow F \ y, F : C \rightarrow D$ 
• fmap_proper:  $\forall x y : C, \text{Proper } (\text{equ} ==> \text{equ}) (@\text{fmap } C \ D \ \_ \ F \ \text{fmap } x \ y)$ 
• fmap_id:  $\forall x : C, \text{fmap } (F := F) (\text{cat\_id } (x := x)) = \text{cat\_id}$ 
• fmap_comp:  $\forall (x y z : C) (f : y \rightarrow z) (g : x \rightarrow y), \text{fmap } (F := F) (f \circ g) = \text{fmap } (F := F) f \circ \text{fmap } (F := F) g$ 
NaturalTransformation (C D) over Category C, Category D extends Functor (source), Functor (target)
ops: F G :  $C \rightarrow D, \text{eta} : \forall x : C, F \ x \rightarrow G \ x$ 
• naturality:  $\forall (x y : C) (f : x \rightarrow y), \text{fmap } (F := G) f \circ \text{eta } x = \text{eta } y \circ \text{fmap } (F := F) f$ 

```

Terminal (O) over Category 0

ops: term (1) : 0, to_term : $\forall x : 0, x \rightarrow 1\text{object}$
 • to_term_unique: $\forall (x : 0) (f : x \rightarrow \text{term}), f == \text{to_term } x$

Initial (O) over Category 0

ops: init (0) : 0, from_init : $\forall x : 0, 0\text{object} \rightarrow x$
 • from_init_unique: $\forall (x : 0) (f : \text{init} \rightarrow x), f == \text{from_init } x$

Products (O) over Category 0

ops: prod_obj (x) : $0 \rightarrow 0 \rightarrow 0$, pi1 : $\forall a b : 0, (a * b)\text{object} \rightarrow a$, pi2 : $\forall a b : 0, (a * b)\text{object} \rightarrow b$, lift (f x x g) : $\forall x a b : 0, x \rightarrow a \rightarrow x \rightarrow b \rightarrow x \rightarrow (a * b)\text{object}$
 • lift_proper: $\forall x a b : 0, \text{Proper } (\text{equ} \Rightarrow \text{equ} \Rightarrow \text{equ}) (@\text{lift } 0 _ _ \text{lift } x a b)$
 • pi1_lift: $\forall (x a b : 0) (f : x \rightarrow a) (g : x \rightarrow b), \text{pi1} \circ \text{lift } f g == f$
 • pi2_lift: $\forall (x a b : 0) (f : x \rightarrow a) (g : x \rightarrow b), \text{pi2} \circ \text{lift } f g == g$
 • lift_unique: $\forall (x a b : 0) (f : x \rightarrow a) (g : x \rightarrow b) (h : x \rightarrow a \times b), \text{pi1} \circ h == f \rightarrow \text{pi2} \circ h == g \rightarrow h == \text{lift } f g$

Coproducts (O) over Category 0

ops: coprod_obj (II) : $0 \rightarrow 0 \rightarrow 0$, inj1 : $\forall a b : 0, a \rightarrow (a + b)\text{object}$, inj2 : $\forall a b : 0, b \rightarrow (a + b)\text{object}$, desc (f III g) : $\forall x a b : 0, a \rightarrow x \rightarrow b \rightarrow x \rightarrow (a + b)\text{object} \rightarrow x$
 • desc_proper: $\forall x a b : 0, \text{Proper } (\text{equ} \Rightarrow \text{equ} \Rightarrow \text{equ}) (@\text{desc } 0 _ _ \text{desc } x a b)$
 • desc_inj1: $\forall (x a b : 0) (f : a \rightarrow x) (g : b \rightarrow x), \text{desc } f g \circ \text{inj1} == f$
 • desc_inj2: $\forall (x a b : 0) (f : a \rightarrow x) (g : b \rightarrow x), \text{desc } f g \circ \text{inj2} == g$
 • desc_unique: $\forall (x a b : 0) (f : a \rightarrow x) (g : b \rightarrow x) (h : a \amalg b \rightarrow x), h \circ \text{inj1} == f \rightarrow h \circ \text{inj2} == g \rightarrow h == \text{desc } f g$

Bicartesian (O) over Category 0 extends Terminal, Initial, Products, Coproducts

ops: term (1) : 0, to_term : $\forall x : 0, x \rightarrow 1\text{object}$, init (0) : 0, from_init : $\forall x : 0, 0\text{object} \rightarrow x$, prod_obj (x) : $0 \rightarrow 0 \rightarrow 0$, pi1 : $\forall a b : 0, (a * b)\text{object} \rightarrow a$, pi2 : $\forall a b : 0, (a * b)\text{object} \rightarrow b$, lift (f x x g) : $\forall x a b : 0, x \rightarrow a \rightarrow x \rightarrow b \rightarrow x \rightarrow (a * b)\text{object}$, coprod_obj (II) : $0 \rightarrow 0 \rightarrow 0$, inj1 : $\forall a b : 0, a \rightarrow (a + b)\text{object}$, inj2 : $\forall a b : 0, b \rightarrow (a + b)\text{object}$, desc (f III g) : $\forall x a b : 0, a \rightarrow x \rightarrow b \rightarrow x \rightarrow (a + b)\text{object} \rightarrow x$

InitialAlgebra (O) over Category 0 extends Functor

ops: build : $F t \rightarrow t$, cata : $\forall x : 0, F x \rightarrow x \rightarrow t \rightarrow x, F : 0 \rightarrow 0, t : 0$
 • cata_proper: $\forall x : 0, \text{Proper } (\text{equ} \Rightarrow \text{equ}) (\text{cata } (F := F) (t := t) x)$
 • cata_build: $\forall (x : 0) (h : F x \rightarrow x), \text{cata } x h \circ \text{build} == h \circ \text{fmap } (\text{cata } x h)$
 • cata_unique: $\forall (x : 0) (h : F x \rightarrow x) (u : t \rightarrow x), u \circ \text{build} == h \circ \text{fmap } u \rightarrow u == \text{cata } x h$

Exponentials (O) over Products 0

ops: exp_obj ($\Rightarrow$) : $0 \rightarrow 0 \rightarrow 0$, eval : $\forall a b : 0, ((a \Rightarrow b) * a)\text{object} \rightarrow b$, curry : $\forall x a b : 0, (x * a)\text{object} \rightarrow b \rightarrow x \rightarrow a \Rightarrow b$
 • curry_proper: $\forall x a b : 0, \text{Proper } (\text{equ} \Rightarrow \text{equ}) (@\text{curry } 0 _ _ _ \text{curry } x a b)$
 • eval_curry: $\forall (x a b : 0) (h : x \times a \rightarrow b), \text{eval} \circ \text{lift } (\text{curry } h \circ \text{pi1}) \text{pi2} == h$
 • curry_unique: $\forall (x a b : 0) (h : x \times a \rightarrow b) (k : x \rightarrow a \Rightarrow b), \text{eval} \circ \text{lift } (k \circ \text{pi1}) \text{pi2} == h \rightarrow k == \text{curry } h$

CartesianClosed (O) over Category 0 extends Terminal, Products, Exponentials

ops: term (1) : 0, to_term : $\forall x : 0, x \rightarrow 1\text{object}$, prod_obj (x) : $0 \rightarrow 0 \rightarrow 0$, pi1 : $\forall a b : 0, (a * b)\text{object} \rightarrow a$, pi2 : $\forall a b : 0, (a * b)\text{object} \rightarrow b$, lift (f x x g) : $\forall x a b : 0, x \rightarrow a \rightarrow x \rightarrow b \rightarrow x \rightarrow (a * b)\text{object}$, exp_obj ($\Rightarrow$) : $0 \rightarrow 0 \rightarrow 0$, eval : $\forall a b : 0, ((a \Rightarrow b) * a)\text{object} \rightarrow b$, curry : $\forall x a b : 0, (x * a)\text{object} \rightarrow b \rightarrow x \rightarrow a \Rightarrow b$

Iso (O) over Category 0

ops: a b : 0, to : $a \rightarrow b$, from : $b \rightarrow a$
 • hom_inv_id: $\text{from} \circ \text{to} == \text{cat_id}$
 • inv_hom_id: $\text{to} \circ \text{from} == \text{cat_id}$

Channel (O) over Category 0

ops: A W : 0, encode : $A \rightarrow W$, decode : $W \rightarrow A$
 • decode_encode: $\text{decode} \circ \text{encode} == \text{cat_id}$

NormedSpace (F V) over VectorSpace F V

ops: norm ($\| v \|$) : $V \rightarrow F$
 • norm_proper: $\text{Proper } (\text{equ} \Rightarrow \text{equ}) (@\text{norm } F V \text{ norm})$
 • norm_nonneg: $\forall v : V, 0 \leq \| v \|$
 • eq_zero_of_norm_eq_zero: $\forall v : V, \| v \| == 0 \rightarrow v == 0$
 • norm_smul: $\forall (a : F) (v : V), \| a * v \| == \text{abs } a * \| v \|$
 • norm_add_le: $\forall u v : V, \| u + v \| \leq \| u \| + \| v \|$

Algebra (F A) over Field F extends VectorSpace, Ring

ops: zero (0) : A, add (+) : $A \rightarrow A \rightarrow A$, neg (- x) : $A \rightarrow A$, one (1) : A, mult (*) : $A \rightarrow A \rightarrow A$, scale (*:) : $F \rightarrow A \rightarrow A$
 • smul_mul_assoc: $\forall (a : F) (x y : A), (a * x) * y == a * (x * y)$
 • mul_smul_comm: $\forall (a : F) (x y : A), x * (a * y) == a * (x * y)$

GeometricAlgebra (F V A) over InnerProductSpace F V extends Linear

ops: vec : $V \rightarrow A$
 • vec_sq: $\forall u : V, \text{vec } u * \text{vec } u == \langle u, u \rangle * 1$

GradeInvolution (F V A) over GeometricAlgebra F V A

ops: involute : $A \rightarrow A$

- involute_proper: Proper (equ ==> equ) (@involute A involute)
- involute_additive: $\forall x y : A, \text{involute } (x + y) == \text{involute } x + \text{involute } y$
- involute_multiplicative: $\forall x y : A, \text{involute } (x * y) == \text{involute } x * \text{involute } y$
- involute_homogeneous: $\forall (a : F) (x : A), \text{involute } (a * x) == a * \text{involute } x$
- involute_one: $\text{involute } (1 : A) == 1$
- involute_involute: $\forall x : A, \text{involute } (\text{involute } x) == x$
- involute_vec: $\forall u : V, \text{involute } (\text{vec } u) == - \text{vec } u$

Limits (F N V) over VectorSpace F V, NatOrder N

ops: tendsto : (N → V) → V → Type, cauchy : (N → V) → Type

- tendsto_proper: $\forall (u u' : N \rightarrow V) (l l' : V), (\forall n : N, u n == u' n) \rightarrow l == l' \rightarrow \text{tendsto } u l \rightarrow \text{tendsto } u' l'$
- tendsto_unique: $\forall (u : N \rightarrow V) (l l' : V), \text{tendsto } u l \rightarrow \text{tendsto } u l' \rightarrow l == l'$
- tendsto_const: $\forall c : V, \text{tendsto } (\lambda _ \Rightarrow c) c$
- tendsto_add: $\forall (u v : N \rightarrow V) (l m : V), \text{tendsto } u l \rightarrow \text{tendsto } v m \rightarrow \text{tendsto } (\lambda n \Rightarrow u n + v n) (l + m)$
- tendsto_neg: $\forall (u : N \rightarrow V) (l : V), \text{tendsto } u l \rightarrow \text{tendsto } (\lambda n \Rightarrow - u n) (- l)$
- tendsto_smul: $\forall (a : F) (u : N \rightarrow V) (l : V), \text{tendsto } u l \rightarrow \text{tendsto } (\lambda n \Rightarrow a * u n) (a * l)$
- tendsto_tail: $\forall (u : N \rightarrow V) (l : V), \text{tendsto } u l \rightarrow \text{tendsto } (\lambda n \Rightarrow u (\text{succ } n)) l$
- cauchy_proper: $\forall u u' : N \rightarrow V, (\forall n : N, u n == u' n) \rightarrow \text{cauchy } u \rightarrow \text{cauchy } u'$
- cauchy_of_tendsto: $\forall (u : N \rightarrow V) (l : V), \text{tendsto } u l \rightarrow \text{cauchy } u$
- cauchy_add: $\forall u v : N \rightarrow V, \text{cauchy } u \rightarrow \text{cauchy } v \rightarrow \text{cauchy } (\lambda n \Rightarrow u n + v n)$
- cauchy_neg: $\forall u : N \rightarrow V, \text{cauchy } u \rightarrow \text{cauchy } (\lambda n \Rightarrow - u n)$
- cauchy_smul: $\forall (a : F) (u : N \rightarrow V), \text{cauchy } u \rightarrow \text{cauchy } (\lambda n \Rightarrow a * u n)$
- cauchy_tail: $\forall u : N \rightarrow V, \text{cauchy } u \rightarrow \text{cauchy } (\lambda n \Rightarrow u (\text{succ } n))$

Complete (F N V) over Limits F N V

ops: lim : $\forall u : N \rightarrow V, \text{cauchy } u \rightarrow V$

- lim_tendsto: $\forall (u : N \rightarrow V) (Hu : \text{cauchy } u), \text{tendsto } u (\text{lim } u Hu)$

Archimedean (F N) over OrderedField F, NatOrder N

ops: of_nat : N → F

- of_nat_proper: Proper (equ ==> equ) of_nat
- of_nat_zero: of_nat 0 == 0
- of_nat_succ: $\forall n : N, \text{of_nat } (\text{succ } n) == \text{of_nat } n + 1$
- archimedean: $\forall q : F, \{n : N \mid q < \text{of_nat } n\}$

Complex (F C) over Algebra F C, Commutative C C mult

ops: imaginary : C, re : C → F, im : C → F

- i_sq: (imaginary : C) * imaginary == - (1 : C)
- re_proper: Proper (equ ==> equ) (@re F C re)
- im_proper: Proper (equ ==> equ) (@im F C im)
- re_additive: $\forall x y : C, \text{re } (x + y) == \text{re } x + \text{re } y$
- im_additive: $\forall x y : C, \text{im } (x + y) == \text{im } x + \text{im } y$
- re_homogeneous: $\forall (a : F) (x : C), \text{re } (a * x) == a * \text{re } x$
- im_homogeneous: $\forall (a : F) (x : C), \text{im } (a * x) == a * \text{im } x$
- re_one: $\text{re } (1 : C) == (1 : F)$
- im_one: $\text{im } (1 : C) == (0 : F)$
- re_i: $\text{re } (\text{imaginary} : C) == (0 : F)$
- im_i: $\text{im } (\text{imaginary} : C) == (1 : F)$
- re_im: $\forall z : C, z == \text{re } z * 1 + \text{im } z * \text{imaginary}$

Frame (F V A IN) over GradeInvolution F V A, Finite I N

ops: top : A → F, basis : I → V

- char_not_two: $(1 + 1 : F) \neq 0$
- basis_proper: Proper (equ ==> equ) basis
- top_proper: Proper (equ ==> equ) (@top F A top)
- top_additive: $\forall x y : A, \text{top } (x + y) == \text{top } x + \text{top } y$
- top_homogeneous: $\forall (a : F) (x : A), \text{top } (a * x) == a * \text{top } x$
- top_pseudoscalar: $\text{top } (\text{natrec } A 1 (\lambda k \text{ acc} \Rightarrow \text{inv } (1 + 1) * (\text{acc} * \text{vec } (\text{basis } (\text{elem } k)) + \text{vec } (\text{basis } (\text{elem } k)) * \text{involute } \text{acc}))) \text{ card} = 1$

Upd (I V)

ops: upd : (I → V) → I → V → I → V

- upd_at: $\forall (vs : I \rightarrow V) (i : I) (x : V) (j : I), j == i \rightarrow \text{upd } vs i x j == x$
- upd_off: $\forall (vs : I \rightarrow V) (i : I) (x : V) (j : I), j \neq i \rightarrow \text{upd } vs i x j == vs j$
- upd_proper: $\forall (vs : I \rightarrow V) (i : I) (x : V), \text{Proper } (\text{equ} ==> \text{equ}) vs \rightarrow \text{Proper } (\text{equ} ==> \text{equ}) (\text{upd } vs i x)$

Multilinear (F I V) over VectorSpace F V

ops: form : (I → V) → F

- form_proper: $\forall vs vs' : I \rightarrow V, \text{Proper } (\text{equ} ==> \text{equ}) vs \rightarrow \text{Proper } (\text{equ} ==> \text{equ}) vs' \rightarrow (\forall i : I, vs i == vs' i) \rightarrow \text{form } vs == \text{form } vs'$
- slot_additive: $\forall (vs : I \rightarrow V) (j : I), \text{Proper } (\text{equ} ==> \text{equ}) vs \rightarrow \text{Additive } V F \text{ add } \text{add } (\lambda x \Rightarrow \text{form } (\text{upd } vs j x))$
- slot_homogeneous: $\forall (vs : I \rightarrow V) (j : I), \text{Proper } (\text{equ} ==> \text{equ}) vs \rightarrow \text{Homogeneous } F V F \text{ mult } (\lambda a \Rightarrow a) (\lambda x \Rightarrow \text{form } (\text{upd } vs j x))$

Alternating (F I V)

ops: form : (I → V) → F

- `alternating`: $\forall (vs : I \rightarrow V) (i\ j : I), \text{Proper } (equ \implies equ) \text{ vs } \rightarrow i \neq j \rightarrow vs\ i == vs\ j \rightarrow \text{form vs} == 0$

Determinant (FIV) over `VectorSpace F V` extends `Multilinear, Alternating`

ops: `det` : $(I \rightarrow V) \rightarrow F$, `basis` : $I \rightarrow V$

- `det_basis`: `det basis == 1`

Sequence (CS)

ops: `step` : $S \rightarrow C \rightarrow S \rightarrow \text{Prop}$

- `elements`: `Equivalence (@equ C equ)`
- `equivalence`: `Equivalence (@equ S equ)`
- `step_proper`: `Proper (equ ==> equ ==> equ ==> iff) (@step C S step)`
- `step_deterministic`: $\forall (s : S) (c\ c' : C) (s'\ s'' : S), \text{step s c s'} \rightarrow \text{step s c' s''} \rightarrow c == c' \wedge s' == s''$

Monad

ops: `ret` : $\forall A : \text{Type}, A \rightarrow P\ A, \text{bind } (>=>) : \forall A\ B : \text{Type}, P\ A \rightarrow (A \rightarrow P\ B) \rightarrow P\ B, P : \text{Type} \rightarrow \text{Type}$

- `equivalence`: $\forall \{ \text{Setoid } A \}, \text{Equivalence } (@\text{equ } (P\ A)) (\text{eqP } A\ \text{eqA})$
- `ret_proper`: $\forall \{ \text{Setoid } A \}, \text{Proper } (equ \implies equ) (\text{ret} : A \rightarrow P\ A)$
- `bind_proper`: $\forall \{ \text{Setoid } A \} \{ \text{Setoid } B \} (p\ p' : P\ A) (f\ f' : A \rightarrow P\ B), \text{Proper } (equ \implies equ) f \rightarrow \text{Proper } (equ \implies equ) f' \rightarrow p == p' \rightarrow (\forall a : A, f\ a == f'\ a) \rightarrow p >=> f == p' >=> f'$
- `bind_ret_l`: $\forall \{ \text{Setoid } A \} \{ \text{Setoid } B \} (a : A) (f : A \rightarrow P\ B), \text{Proper } (equ \implies equ) f \rightarrow (\text{ret } a : P\ A) >=> f == f\ a$
- `bind_ret_r`: $\forall \{ \text{Setoid } A \} (p : P\ A), p >=> \text{ret} == p$
- `bind_assoc`: $\forall \{ \text{Setoid } A \} \{ \text{Setoid } B \} \{ \text{Setoid } D \} (p : P\ A) (f : A \rightarrow P\ B) (g : B \rightarrow P\ D), \text{Proper } (equ \implies equ) f \rightarrow \text{Proper } (equ \implies equ) g \rightarrow (p >=> f) >=> g == p >=> (\lambda a \rightarrow f\ a >=> g)$

MonadPlus extends `Monad`

ops: `ret` : $\forall A : \text{Type}, A \rightarrow P\ A, \text{bind } (>=>) : \forall A\ B : \text{Type}, P\ A \rightarrow (A \rightarrow P\ B) \rightarrow P\ B, \text{fail} : \forall A : \text{Type}, P\ A, \text{alt } (<|>) : \forall A : \text{Type}, P\ A \rightarrow P\ A \rightarrow P\ A, P : \text{Type} \rightarrow \text{Type}$

- `alt_proper`: $\forall \{ \text{Setoid } A \}, \text{Proper } (equ \implies equ \implies equ) (\text{alt} : P\ A \rightarrow P\ A \rightarrow P\ A)$
- `alt_assoc`: $\forall \{ \text{Setoid } A \} (p\ q\ r : P\ A), (p <|> q) <|> r == p <|> (q <|> r)$
- `alt_fail_l`: $\forall \{ \text{Setoid } A \} (p : P\ A), (\text{fail} : P\ A) <|> p == p$
- `alt_fail_r`: $\forall \{ \text{Setoid } A \} (p : P\ A), p <|> \text{fail} == p$
- `bind_fail_l`: $\forall \{ \text{Setoid } A \} \{ \text{Setoid } B \} (f : A \rightarrow P\ B), \text{Proper } (equ \implies equ) f \rightarrow (\text{fail} : P\ A) >=> f == \text{fail}$
- `bind_alt_l`: $\forall \{ \text{Setoid } A \} \{ \text{Setoid } B \} (p\ q : P\ A) (f : A \rightarrow P\ B), \text{Proper } (equ \implies equ) f \rightarrow (p <|> q) >=> f == (p >=> f) <|> (q >=> f)$

Parser (CS)

ops: `ret` : $\forall A : \text{Type}, A \rightarrow P\ A, \text{bind } (>=>) : \forall A\ B : \text{Type}, P\ A \rightarrow (A \rightarrow P\ B) \rightarrow P\ B, \text{fail} : \forall A : \text{Type}, P\ A, \text{alt } (<|>) : \forall A : \text{Type}, P\ A \rightarrow P\ A \rightarrow P\ A, \text{item} : P\ C, \text{parses} : \forall A : \text{Type}, \text{EqRel } A \rightarrow P\ A \rightarrow S \rightarrow A \rightarrow S \rightarrow \text{Prop}, P : \text{Type} \rightarrow \text{Type}$

- `parser_equ`: $\forall \{ \text{Setoid } A \} (p\ q : P\ A), p == q \leftrightarrow (\forall (s\ s' : S) (a : A), \text{parses } p\ s\ a\ s' \leftrightarrow \text{parses } q\ s\ a\ s')$
- `parses_proper`: $\forall \{ \text{Setoid } A \} (p : P\ A), \text{Proper } (equ \implies equ \implies equ) (\text{parses } p)$
- `parses_ret`: $\forall \{ \text{Setoid } A \} (a\ a' : A) (s\ s' : S), \text{parses } (\text{ret } a : P\ A)\ s\ a'\ s' \leftrightarrow a' == a \wedge s' == s$
- `parses_bind`: $\forall \{ \text{Setoid } A \} \{ \text{Setoid } B \} (p : P\ A) (f : A \rightarrow P\ B) (s\ s' : S) (b : B), \text{Proper } (equ \implies equ) f \rightarrow (\text{parses } (p >=> f : P\ B)\ s\ b\ s' \leftrightarrow \text{exists } (a : A) (s' : S), \text{parses } p\ s\ a\ s' \wedge \text{parses } (f\ a)\ s'\ b\ s')$
- `parses_fail`: $\forall \{ \text{Setoid } A \} (s\ s' : S) (a : A), \sim \text{parses } (\text{fail} : P\ A)\ s\ a\ s'$
- `parses_alt`: $\forall \{ \text{Setoid } A \} (p\ q : P\ A) (s\ s' : S) (a : A), \text{parses } (p <|> q)\ s\ a\ s' \leftrightarrow \text{parses } p\ s\ a\ s' \wedge \text{parses } q\ s\ a\ s'$
- `parses_item`: $\forall (s\ s' : S) (c : C), \text{parses } (\text{item} : P\ C)\ s\ c\ s' \leftrightarrow \text{step s c s'}$

CPSLambda (NTSb) over `LambdaCalculus N T Sb`

ops: `trivial` : $T \rightarrow \text{Prop}$, `tail` : $T \rightarrow \text{Prop}$, `cps` : $T \rightarrow T$, `psi` : $T \rightarrow T$, `evaluates` : $T \rightarrow T \rightarrow \text{Prop}$

- `trivial_var`: $\forall n : N, \text{trivial } (\text{var } n)$
- `trivial_lam`: $\forall b : T, \text{tail } b \rightarrow \text{trivial } (\text{lam } b)$
- `tail_of_trivial`: $\forall v : T, \text{trivial } v \rightarrow \text{tail } v$
- `tail_call`: $\forall f\ v : T, \text{trivial } f \rightarrow \text{trivial } v \rightarrow \text{tail } (\text{app } f\ v)$
- `tail_call2`: $\forall f\ v\ w : T, \text{trivial } f \rightarrow \text{trivial } v \rightarrow \text{trivial } w \rightarrow \text{tail } (\text{app } (\text{app } f\ v)\ w)$
- `cps_trivial`: $\forall t : T, \text{trivial } (\text{cps } t)$
- `cps_value`: $\forall v\ k : T, \text{trivial } v \rightarrow \text{app } (\text{cps } v)\ k == \text{app } k\ (\text{psi } v)$
- `cps_runs`: $\forall l\ v\ k : T, \text{evaluates } l\ v \rightarrow \text{app } (\text{cps } l)\ k == \text{app } k\ (\text{psi } v)$

Defunctionalization (NTSbK) over `CPSLambda N T Sb`

ops: `is_value` : $T \rightarrow \text{Prop}$, `halt` : K , `karg` : $T \rightarrow K \rightarrow K$, `kfun` : $T \rightarrow K \rightarrow K$, `reify` : $K \rightarrow T$, `druns` : $T \rightarrow K \rightarrow T \rightarrow \text{Prop}$

- `value_lam`: $\forall b : T, \text{is_value } (\text{lam } b)$
- `value_evaluates`: $\forall v : T, \text{is_value } v \rightarrow \text{evaluates } v\ v$
- `reify_halt`: $\forall t : T, \text{app } (\text{reify } \text{halt})\ t == t$
- `reify_karg`: $\forall (f\ a : T) (k : K), \text{app } (\text{cps } (\text{app } f\ a))\ (\text{reify } k) == \text{app } (\text{cps } f)\ (\text{reify } (\text{karg } a\ k))$
- `reify_kfun`: $\forall (a\ f : T) (k : K), \text{is_value } f \rightarrow \text{app } (\text{reify } (\text{karg } a\ k))\ (\text{psi } f) == \text{app } (\text{cps } a)\ (\text{reify } (k\lambda f\ k))$
- `reify_call`: $\forall (b\ v : T) (k : K), \text{is_value } v \rightarrow \text{app } (\text{reify } (k\lambda (\text{lam } b)\ k))\ (\text{psi } v) == \text{app } (\text{cps } (b.[v \text{.} \text{sub_id}]))\ (\text{reify } k)$
- `druns_sound`: $\forall (t\ v : T) (k : K), \text{druns } t\ k\ v \rightarrow \text{app } (\text{cps } t)\ (\text{reify } k) == \text{psi } v$
- `druns_complete`: $\forall l\ v : T, \text{evaluates } l\ v \rightarrow \text{druns } l\ \text{halt } v$

Graded (NFA) over `Naturals N, Algebra F A`

ops: `grade_part` : $((x\)_k) : N \rightarrow A \rightarrow A$, `top_grade` : N

- `grade_linear`: $\forall k : N, \text{Linear } F\ A\ A\ (\text{grade_part } k)$
- `grade_index_proper`: $\forall (k\ k' : N) (x : A), k == k' \rightarrow \{x\}_k == \{x\}_{k'}$

- $\text{grade_idempotent} : \forall (k : \mathbb{N}) (x : A), \langle \langle x \rangle_k \rangle_k = \langle x \rangle_k$
- $\text{grade_orthogonal} : \forall (k \ l : \mathbb{N}) (x : A), k \neq l \rightarrow \langle \langle x \rangle_l \rangle_k = 0$
- $\text{grade_complete} : \forall x : A, \text{natrec } A \ 0 \ (\lambda k \ \text{acc} \Rightarrow \text{acc} + \langle x \rangle_k) (\text{succ } \text{top_grade}) = x$

Reversal (A) over Ring A

ops: reverse : A → A

- reverse_proper: Proper (equ ==> equ) (@reverse A reverse)
- reverse_additive: $\forall x \ y : A, \text{reverse } (x + y) = \text{reverse } x + \text{reverse } y$
- reverse_anti: $\forall x \ y : A, \text{reverse } (x * y) = \text{reverse } y * \text{reverse } x$
- reverse_involutive: $\forall x : A, \text{reverse } (\text{reverse } x) = x$

GradedGA (NFV A) over GradeInvolution F V A, Naturals N extends Graded, Reversal

ops: grade_part ((x) k) : N → A → A, scalar_part : A → F, reverse : A → A, top_grade : N

- one_grade_zero: $((1 : A))_0 = 1$
- grade_zero_scalar: $\forall x : A, \langle x \rangle_0 = \text{scalar_part } x * 1$
- vec_grade_one: $\forall u : V, (\text{vec } u)_0 (\text{succ } 0) = \text{vec } u$
- vec_mul_grade_zero: $\forall (u : V) (x : A), \text{vec } u * \langle x \rangle_0 = (\text{vec } u * \langle x \rangle_0) (\text{succ } 0)$
- vec_mul_grade_succ: $\forall (u : V) (x : A) (k : \mathbb{N}), \text{vec } u * \langle x \rangle_k (\text{succ } k) = (\text{vec } u * \langle x \rangle_k) (\text{succ } k) + (\text{vec } u * \langle x \rangle_k) (\text{succ } k)$
- involute_grade: $\forall (k : \mathbb{N}) (x : A), \text{involute } \langle x \rangle_k = (\text{involute } x)_k$
- reverse_grade: $\forall (k : \mathbb{N}) (x : A), \text{reverse } \langle x \rangle_k = \langle \text{reverse } x \rangle_k$
- reverse_homogeneous: $\forall (a : F) (x : A), \text{reverse } (a * x) = a * \text{reverse } x$
- reverse_vec: $\forall u : V, \text{reverse } (\text{vec } u) = \text{vec } u$
- reverse_involute: $\forall x : A, \text{reverse } (\text{involute } x) = \text{involute } (\text{reverse } x)$

Rotor (FV A) over GradeInvolution F V A

ops: R : A

- rotor_even: involute R = R
- rotor_unit_r: R * reverse R = 1
- rotor_unit_l: reverse R * R = 1

Basis (FIV N) over VectorSpace F V, Finite I N

ops: coords : V → I → F, basis : I → V

- basis_proper: Proper (equ ==> equ) basis
- coords_proper: Proper (equ ==> equ ==> equ) (@coords F I V coords)
- coords_additive: $\forall i : I, \text{Additive } V \ F \ \text{add } \text{add } (\lambda v \Rightarrow \text{coords } v \ i)$
- coords_homogeneous: $\forall i : I, \text{Homogeneous } F \ V \ F \ \text{mult } (\lambda a \Rightarrow a) (\lambda v \Rightarrow \text{coords } v \ i)$
- reconstruction: $\forall v : V, \text{natrec } V \ 0 \ (\lambda k \ \text{acc} \Rightarrow \text{acc} + \text{coords } v \ (\text{elem } k) *) : \text{basis } (\text{elem } k) \ \text{card} = v$
- coords_basis: $\forall i \ j : I, \text{coords } (\text{basis } j) \ i = (\text{if } \text{EquDec.equ_dec } i \ j \text{ then } (1 : F) \text{ else } 0)$

LambdaCalculus (NT Sb) over Naturals N

ops: var : N → T, lam : T → T, app : T → T → T, substitute (t .[s]) : T → Sb → T, sub_id : Sb, sub_shift : Sb, sub_cons (.) : T → Sb → Sb, sub_up : Sb → Sb, reduces (→*) : T → T → Prop

- equ_join: $\forall t \ u : T, t = u \leftrightarrow (\text{exists } w : T, t \rightarrow^* w \wedge u \rightarrow^* w)$
- reduces_refl: $\forall t : T, t \rightarrow^* t$
- reduces_trans: $\forall t \ u \ v : T, t \rightarrow^* u \rightarrow u \rightarrow^* v \rightarrow t \rightarrow^* v$
- reduces_lam: $\forall b \ b' : T, b \rightarrow^* b' \rightarrow \text{lam } b \rightarrow^* \text{lam } b'$
- reduces_app: $\forall f \ f' \ a \ a' : T, f \rightarrow^* f' \rightarrow a \rightarrow^* a' \rightarrow \text{app } f \ a \rightarrow^* \text{app } f' \ a'$
- reduces_subst: $\forall (t \ t' : T) (s : Sb), t \rightarrow^* t' \rightarrow t.[s] \rightarrow^* t'.[s]$
- beta: $\forall b \ u : T, \text{app } (\text{lam } b) \ u \rightarrow^* b.[u : \text{sub_id}]$
- var_injective: $\forall n \ m : N, (\text{var } n : T) = \text{var } m \rightarrow n = m$
- subst_id: $\forall t : T, t.[\text{sub_id}] = t$
- var_zero_cons: $\forall (u : T) (s : Sb), (\text{var } 0 : T).[u : s] = u$
- var_succ_cons: $\forall (n : N) (u : T) (s : Sb), (\text{var } (\text{succ } n) : T).[u : s] = (\text{var } n : T).[s]$
- var_shift: $\forall n : N, (\text{var } n : T).[sub_shift] = \text{var } (\text{succ } n)$
- var_zero_up: $\forall s : Sb, (\text{var } 0 : T).[sub_up \ s] = \text{var } 0$
- var_succ_up: $\forall (n : N) (s : Sb), (\text{var } (\text{succ } n) : T).[sub_up \ s] = ((\text{var } n : T).[s]).[sub_shift]$
- app_subst: $\forall (f \ a : T) (s : Sb), (\text{app } f \ a).[s] = \text{app } (f.[s]) \ (a.[s])$
- lam_subst: $\forall (b : T) (s : Sb), (\text{lam } b).[s] = \text{lam } (b.[sub_up \ s])$
- shift_cons: $\forall (t \ u : T) (s : Sb), (t.[sub_shift]).[u : s] = t.[s]$
- shift_up: $\forall (t : T) (s : Sb), (t.[sub_shift]).[sub_up \ s] = (t.[s]).[sub_shift]$
- confluence: $\forall t \ u \ v : T, t \rightarrow^* u \rightarrow u \rightarrow^* v \rightarrow \text{exists } w : T, u \rightarrow^* w \wedge v \rightarrow^* w$

StlcType (O K) over Bicartesian 0 extends InitialAlgebra

ops: build : shape den b Ty → Ty, cata : $\forall x : 0, \text{shape den } b \ x \rightarrow x \rightarrow \text{Ty} \rightarrow x, \text{den} : K \rightarrow 0, b : K, \text{Ty} : 0$

Stlc (O K) over Bicartesian 0 extends InitialAlgebra

ops: build : shape den n ty T → T, cata : $\forall x : 0, \text{shape den } n \ \text{ty } x \rightarrow x \rightarrow T \rightarrow x, \text{den} : K \rightarrow 0, n \ \text{ty} : K, T : 0$

StlcCalculus (NTy T Sb) over Naturals N

ops: var : N → T, tylam : Ty → T → T, app : T → T → T, substitute (t .[s]) : T → Sb → T, sub_id : Sb, sub_shift : Sb, sub_cons (.) : T → Sb → Sb, sub_up : Sb → Sb, reduces (→*) : T → T → Prop

- equ_join: $\forall t \ u : T, t = u \leftrightarrow (\text{exists } w : T, t \rightarrow^* w \wedge u \rightarrow^* w)$
- reduces_refl: $\forall t : T, t \rightarrow^* t$
- reduces_trans: $\forall t \ u \ v : T, t \rightarrow^* u \rightarrow u \rightarrow^* v \rightarrow t \rightarrow^* v$
- reduces tylam: $\forall (A : \text{Ty}) (b \ b' : T), b \rightarrow^* b' \rightarrow \text{tylam } A \ b \rightarrow^* \text{tylam } A \ b'$

- $\text{reduces_app} : \forall f f' a a' : T, f \rightarrow^* f' \rightarrow a \rightarrow^* a' \rightarrow \text{app } f a \rightarrow^* \text{app } f' a'$
- $\text{reduces_subst} : \forall (t t' : T) (s : Sb), t \rightarrow^* t' \rightarrow t.[s] \rightarrow^* t'.[s]$
- $\text{tylam_proper} : \forall (A A' : Ty) (b : T), A == A' \rightarrow \text{tylam } A b == \text{tylam } A' b$
- $\text{beta} : \forall (A : Ty) (b u : T), \text{app } (\text{tylam } A b) u \rightarrow^* b.[u \text{ : sub_id}]$
- $\text{var_injective} : \forall n m : N, (\text{var } n : T) == \text{var } m \rightarrow n == m$
- $\text{subst_id} : \forall t : T, t.[\text{sub_id}] == t$
- $\text{var_zero_cons} : \forall (u : T) (s : Sb), (\text{var } 0 : T).[u \text{ : s}] == u$
- $\text{var_succ_cons} : \forall (n : N) (u : T) (s : Sb), (\text{var } (\text{succ } n) : T).[u \text{ : s}] == (\text{var } n : T).[s]$
- $\text{var_shift} : \forall n : N, (\text{var } n : T).[sub_shift] == \text{var } (\text{succ } n)$
- $\text{var_zero_up} : \forall s : Sb, (\text{var } 0 : T).[sub_up s] == \text{var } 0$
- $\text{var_succ_up} : \forall (n : N) (s : Sb), (\text{var } (\text{succ } n) : T).[sub_up s] == ((\text{var } n : T).[s]).[sub_shift]$
- $\text{app_subst} : \forall (f a : T) (s : Sb), (\text{app } f a).[s] == \text{app } (f.[s]) (a.[s])$
- $\text{tylam_subst} : \forall (A : Ty) (b : T) (s : Sb), (\text{tylam } A b).[s] == \text{tylam } A (b.[sub_up s])$
- $\text{shift_cons} : \forall (t u : T) (s : Sb), (t.[sub_shift]).[u \text{ : s}] == t.[s]$
- $\text{shift_up} : \forall (t : T) (s : Sb), (t.[sub_shift]).[sub_up s] == (t.[s]).[sub_shift]$
- $\text{confluence} : \forall t u v : T, t \rightarrow^* u \rightarrow t \rightarrow^* v \rightarrow \text{exists } w : T, u \rightarrow^* w \wedge v \rightarrow^* w$

StlcTyping (N Ty T Sb G) over StlcCalculus N Ty T Sb, NatOrder N leN

ops: $\text{lookup} : G \rightarrow N \rightarrow Ty, \text{extend} : Ty \rightarrow G \rightarrow G, \text{empty} : G, \text{bound} : G \rightarrow N, \text{typing} : G \rightarrow T \rightarrow Ty \rightarrow Prop$

- $\text{typing_proper} : \forall (g : G) (t : T) (A A' : Ty), A == A' \rightarrow \text{typing } g t A \rightarrow \text{typing } g t A'$
- $\text{typed_var} : \forall (g : G) (n : N) (A : Ty), \text{succ } n \leq \text{bound_g} \rightarrow \text{lookup_g } n == A \rightarrow \text{typing } g (\text{var } n) A$
- $\text{typed_tylam} : \forall (g : G) (A C : Ty) (b : T), \text{typing } (\text{extend_A } g) b C \rightarrow \text{typing } g (\text{tylam } A b) (A \rightarrow C)$
- $\text{typed_app} : \forall (g : G) (A C : Ty) (f a : T), \text{typing } g f (A \rightarrow C) \rightarrow \text{typing } g a A \rightarrow \text{typing } g (\text{app } f a) C$
- $\text{lookup_extend_zero} : \forall (g : G) (A : Ty), \text{lookup_ } (\text{extend_A } g) 0 == A$
- $\text{lookup_extend_succ} : \forall (g : G) (A : Ty) (n : N), \text{lookup_ } (\text{extend_A } g) (\text{succ } n) == \text{lookup_g } n$
- $\text{bound_extend} : \forall (g : G) (A : Ty), \text{bound_ } (\text{extend_A } g) == \text{succ } (\text{bound_g})$
- $\text{bound_empty} : \text{bound_empty} == 0$
- $\text{preservation} : \forall (g : G) (t t' : T) (A : Ty), \text{typing } g t A \rightarrow t \rightarrow^* t' \rightarrow \text{typing } g t' A$
- $\text{typed_reduces} : \forall (t : T) (A C : Ty), \text{typing } \text{empty_t } (A \rightarrow C) \rightarrow \text{exists } (A' : Ty) (b : T), t \rightarrow^* \text{tylam } A' b$

Signature (K S)

ops: $\text{zero } (0) : S, \text{one } (1) : S, \text{self} : S, \text{const} : K \rightarrow S, \text{add } (+) : S \rightarrow S \rightarrow S, \text{mult } (*) : S \rightarrow S \rightarrow S, \text{poly_rec} : \forall B : Type, B \rightarrow B \rightarrow B \rightarrow (K \rightarrow B) \rightarrow (B \rightarrow B \rightarrow B) \rightarrow (B \rightarrow B \rightarrow B) \rightarrow S \rightarrow B$

- $\text{rec_zero} : \forall (B : Type) (z o s : B) (c : K \rightarrow B) (p t : B \rightarrow B \rightarrow B), \text{poly_rec } B z o s c p t (0 : S) = z$
- $\text{rec_one} : \forall (B : Type) (z o s : B) (c : K \rightarrow B) (p t : B \rightarrow B \rightarrow B), \text{poly_rec } B z o s c p t (1 : S) = o$
- $\text{rec_self} : \forall (B : Type) (z o s : B) (c : K \rightarrow B) (p t : B \rightarrow B \rightarrow B), \text{poly_rec } B z o s c p t (\text{self} : S) = s$
- $\text{rec_const} : \forall (B : Type) (z o s : B) (c : K \rightarrow B) (p t : B \rightarrow B \rightarrow B) (k : K), \text{poly_rec } B z o s c p t (\text{const } k : S) = c k$
- $\text{rec_plus} : \forall (B : Type) (z o s : B) (c : K \rightarrow B) (p t : B \rightarrow B \rightarrow B) (a b : S), \text{poly_rec } B z o s c p t (a + b) = p (\text{poly_rec } B z o s c p t a) (\text{poly_rec } B z o s c p t b)$
- $\text{rec_times} : \forall (B : Type) (z o s : B) (c : K \rightarrow B) (p t : B \rightarrow B \rightarrow B) (a b : S), \text{poly_rec } B z o s c p t (a * b) = t (\text{poly_rec } B z o s c p t a) (\text{poly_rec } B z o s c p t b)$
- $\text{rec_unique} : \forall (B : Type) (z o s : B) (c : K \rightarrow B) (p t : B \rightarrow B \rightarrow B) (u : S \rightarrow B), u 0 = z \rightarrow u 1 = o \rightarrow u \text{ self} = s \rightarrow (\forall k : K, u (\text{const } k) = c k) \rightarrow (\forall a b : S, u (a + b) = p (u a) (u b)) \rightarrow (\forall a b : S, u (a * b) = t (u a) (u b)) \rightarrow \forall q : S, u q = \text{poly_rec } B z o s c p t q$

ListRec (X L)

ops: $\text{empty} : L, \text{prepend} : X \rightarrow L \rightarrow L, \text{list_rec} : \forall B : Type, B \rightarrow (X \rightarrow B \rightarrow B) \rightarrow L \rightarrow B$

- $\text{list_rec_proper} : \forall (B : Type) (eB : EqRel B) (EB : Equivalence eB) (n : B) (c : X \rightarrow B \rightarrow B), \text{Proper } (\text{equ} ==> \text{equ} ==> \text{equ}) c \rightarrow \text{Proper } (\text{equ} ==> \text{equ}) (\text{list_rec } B n c : L \rightarrow B)$
- $\text{list_rec_empty} : \forall (B : Type) (eB : EqRel B) (EB : Equivalence eB) (n : B) (c : X \rightarrow B \rightarrow B), \text{list_rec } B n c (\text{empty} : L) == n$
- $\text{list_rec_prepend} : \forall (B : Type) (eB : EqRel B) (EB : Equivalence eB) (n : B) (c : X \rightarrow B \rightarrow B) (x : X) (l : L), \text{list_rec } B n c (\text{prepend } x l) == c x (\text{list_rec } B n c l)$

TermRec (N T)

ops: $\text{var} : N \rightarrow T, \text{lam} : T \rightarrow T, \text{app} : T \rightarrow T \rightarrow T, \text{termrec} : \forall B : Type, (N \rightarrow B) \rightarrow (B \rightarrow B) \rightarrow (B \rightarrow B \rightarrow B) \rightarrow T \rightarrow B$

- $\text{termrec_proper} : \forall (B : Type) (eB : EqRel B) (EB : Equivalence eB) (v : N \rightarrow B) (l : B \rightarrow B) (a : B \rightarrow B \rightarrow B), \text{Proper } (\text{equ} ==> \text{equ}) v \rightarrow \text{Proper } (\text{equ} ==> \text{equ}) l \rightarrow \text{Proper } (\text{equ} ==> \text{equ} ==> \text{equ}) a \rightarrow \text{Proper } (\text{equ} ==> \text{equ}) (\text{termrec } B v l a : T \rightarrow B)$
- $\text{termrec_var} : \forall (B : Type) (eB : EqRel B) (EB : Equivalence eB) (v : N \rightarrow B) (l : B \rightarrow B) (a : B \rightarrow B \rightarrow B) (n : N), \text{termrec } B v l a (\text{var } n : T) == v n$
- $\text{termrec_lam} : \forall (B : Type) (eB : EqRel B) (EB : Equivalence eB) (v : N \rightarrow B) (l : B \rightarrow B) (a : B \rightarrow B \rightarrow B) (b : T), \text{termrec } B v l a (\text{lam } b) == l (\text{termrec } B v l a b)$
- $\text{termrec_app} : \forall (B : Type) (eB : EqRel B) (EB : Equivalence eB) (v : N \rightarrow B) (l : B \rightarrow B) (a : B \rightarrow B \rightarrow B) (f u : T), \text{termrec } B v l a (\text{app } f u) == a (\text{termrec } B v l a f) (\text{termrec } B v l a u)$

NamedTermRec (K N T)

ops: $\text{nvar} : K \rightarrow T, \text{num} : N \rightarrow T, \text{nlam} : K \rightarrow T \rightarrow T, \text{app} : T \rightarrow T \rightarrow T, \text{ntermrec} : \forall B : Type, (K \rightarrow B) \rightarrow (N \rightarrow B) \rightarrow (K \rightarrow B \rightarrow B) \rightarrow (B \rightarrow B \rightarrow B) \rightarrow T \rightarrow B$

- $\text{ntermrec_proper} : \forall (B : Type) (eB : EqRel B) (EB : Equivalence eB) (v : K \rightarrow B) (c : N \rightarrow B) (l : K \rightarrow B \rightarrow B) (a : B \rightarrow B \rightarrow B), \text{Proper } (\text{equ} ==> \text{equ}) v \rightarrow \text{Proper } (\text{equ} ==> \text{equ}) c \rightarrow \text{Proper } (\text{equ} ==> \text{equ} ==> \text{equ}) a \rightarrow \text{Proper } (\text{equ} ==> \text{equ}) (\text{ntermrec } B v c l a : T \rightarrow B)$

- **ntermrec_var**: $\forall (B : \text{Type}) (eB : \text{EqRel } B) (EB : \text{Equivalence } eB) (v : K \rightarrow B) (c : N \rightarrow B) (l : K \rightarrow B \rightarrow B) (a : B \rightarrow B \rightarrow B) (x : K), \text{ntermrec } B \ v \ c \ l \ a \ (\text{nvar } x : T) == v \ x$
- **ntermrec_num**: $\forall (B : \text{Type}) (eB : \text{EqRel } B) (EB : \text{Equivalence } eB) (v : K \rightarrow B) (c : N \rightarrow B) (l : K \rightarrow B \rightarrow B) (a : B \rightarrow B \rightarrow B) (n : N), \text{ntermrec } B \ v \ c \ l \ a \ (\text{num } n : T) == c \ n$
- **ntermrec_lam**: $\forall (B : \text{Type}) (eB : \text{EqRel } B) (EB : \text{Equivalence } eB) (v : K \rightarrow B) (c : N \rightarrow B) (l : K \rightarrow B \rightarrow B) (a : B \rightarrow B \rightarrow B) (x : K) (b : T), \text{ntermrec } B \ v \ c \ l \ a \ (\text{n\text{lam } x } b) == l \ x \ (\text{ntermrec } B \ v \ c \ l \ a \ b)$
- **ntermrec_app**: $\forall (B : \text{Type}) (eB : \text{EqRel } B) (EB : \text{Equivalence } eB) (v : K \rightarrow B) (c : N \rightarrow B) (l : K \rightarrow B \rightarrow B) (a : B \rightarrow B \rightarrow B) (f \ u : T), \text{ntermrec } B \ v \ c \ l \ a \ (\text{app } f \ u) == a \ (\text{ntermrec } B \ v \ c \ l \ a \ f) \ (\text{ntermrec } B \ v \ c \ l \ a \ u)$

TypedTermRec (N Ty T)

ops: **var** : $N \rightarrow T$, **tylam** : $Ty \rightarrow T \rightarrow T$, **app** : $T \rightarrow T \rightarrow T$, **stermrec** : $\forall B : \text{Type}, (N \rightarrow B) \rightarrow (Ty \rightarrow B \rightarrow B) \rightarrow (B \rightarrow B \rightarrow B) \rightarrow T \rightarrow B$

- **stermrec_proper**: $\forall (B : \text{Type}) (eB : \text{EqRel } B) (EB : \text{Equivalence } eB) (v : N \rightarrow B) (l : Ty \rightarrow B \rightarrow B) (a : B \rightarrow B \rightarrow B), \text{Proper } (\text{equ} ==> \text{equ}) \ v \rightarrow \text{Proper } (\text{equ} ==> \text{equ} ==> \text{equ}) \ l \rightarrow \text{Proper } (\text{equ} ==> \text{equ} ==> \text{equ}) \ a \rightarrow \text{Proper } (\text{equ} ==> \text{equ}) \ (\text{stermrec } B \ v \ l \ a : T \rightarrow B)$
- **stermrec_var**: $\forall (B : \text{Type}) (eB : \text{EqRel } B) (EB : \text{Equivalence } eB) (v : N \rightarrow B) (l : Ty \rightarrow B \rightarrow B) (a : B \rightarrow B \rightarrow B) (n : N), \text{stermrec } B \ v \ l \ a \ (\text{var } n : T) == v \ n$
- **stermrec_tylam**: $\forall (B : \text{Type}) (eB : \text{EqRel } B) (EB : \text{Equivalence } eB) (v : N \rightarrow B) (l : Ty \rightarrow B \rightarrow B) (a : B \rightarrow B \rightarrow B) (A : Ty) (b : T), \text{stermrec } B \ v \ l \ a \ (\text{tylam } A \ b) == l \ A \ (\text{stermrec } B \ v \ l \ a \ b)$
- **stermrec_app**: $\forall (B : \text{Type}) (eB : \text{EqRel } B) (EB : \text{Equivalence } eB) (v : N \rightarrow B) (l : Ty \rightarrow B \rightarrow B) (a : B \rightarrow B \rightarrow B) (f \ u : T), \text{stermrec } B \ v \ l \ a \ (\text{app } f \ u) == a \ (\text{stermrec } B \ v \ l \ a \ f) \ (\text{stermrec } B \ v \ l \ a \ u)$

Derivative (F U V) over VectorSpace F U

ops: **differential** : $(U \rightarrow V) \rightarrow U \rightarrow (U \rightarrow V) \rightarrow \text{Prop}$

- **differential_proper**: $\forall (f \ f' : U \rightarrow V) (a \ a' : U) (L \ L' : U \rightarrow V), (\forall x : U, f \ x == f' \ x) \rightarrow a == a' \rightarrow (\forall h : U, L \ h == L' \ h) \rightarrow \text{differential } f \ a \ L \rightarrow \text{differential } f' \ a' \ L'$
- **differential_linear**: $\forall (f : U \rightarrow V) (a : U) (L : U \rightarrow V), \text{differential } f \ a \ L \rightarrow \text{Linear } F \ U \ V \ L$
- **differential_unique**: $\forall (f : U \rightarrow V) (a : U) (L \ L' : U \rightarrow V), \text{differential } f \ a \ L \rightarrow \text{differential } f \ a \ L' \rightarrow \forall h : U, L \ h == L' \ h$
- **differential_const**: $\forall (v : V) (a : U), \text{differential } (\lambda _ \Rightarrow v) \ a \ (\lambda _ \Rightarrow 0)$
- **differential_of_linear**: $\forall f : U \rightarrow V, \text{Linear } F \ U \ V \ f \rightarrow \forall a : U, \text{differential } f \ a \ f$
- **differential_add**: $\forall (f \ g : U \rightarrow V) (a : U) (L \ Lg : U \rightarrow V), \text{differential } f \ a \ L \rightarrow \text{differential } g \ a \ Lg \rightarrow \text{differential } (\lambda \ x \Rightarrow f \ x + g \ x) \ a \ (\lambda \ h \Rightarrow L \ h + Lg \ h)$
- **differential_smul**: $\forall (c : F) (f : U \rightarrow V) (a : U) (L : U \rightarrow V), \text{differential } f \ a \ L \rightarrow \text{differential } (\lambda \ x \Rightarrow c * f \ x) \ a \ (\lambda \ h \Rightarrow c * L \ h)$

ChainRule (U V W)

ops: **differential** : $(U \rightarrow V) \rightarrow U \rightarrow (U \rightarrow V) \rightarrow \text{Prop}$, **differential** : $(V \rightarrow W) \rightarrow V \rightarrow (V \rightarrow W) \rightarrow \text{Prop}$, **differential** : $(U \rightarrow W) \rightarrow U \rightarrow (U \rightarrow W) \rightarrow \text{Prop}$

- **differential_comp**: $\forall (f : U \rightarrow V) (g : V \rightarrow W) (a : U) (L : U \rightarrow V) (Lg : V \rightarrow W), \text{differential } f \ a \ L \rightarrow \text{differential } g \ (f \ a) \ Lg \rightarrow \text{differential } (\lambda \ x \Rightarrow g \ (f \ x)) \ a \ (\lambda \ h \Rightarrow Lg \ (L \ h))$

TableSpace (F I T) over VectorSpace F T

- **get_zero**: $\forall i : I, \text{get } 0 \ i == 0$
- **get_add**: $\forall (s \ t : T) (i : I), \text{get } (s + t) \ i == \text{get } s \ i + \text{get } t \ i$
- **get_smul**: $\forall (a : F) (t : T) (i : I), \text{get } (a * t) \ i == a * \text{get } t \ i$

Tree (A Tr)

ops: **tip** : $A \rightarrow \text{Tr}$, **fork** : $\text{Tr} \rightarrow \text{Tr} \rightarrow \text{Tr}$

- **equivalence**: $\text{Equivalence } (@\text{equ } \text{Tr } \text{equ})$
- **tip_proper**: $\text{Proper } (\text{equ} ==> \text{equ}) \ (\text{tip} : A \rightarrow \text{Tr})$
- **fork_proper**: $\text{Proper } (\text{equ} ==> \text{equ} ==> \text{equ}) \ (\text{fork} : \text{Tr} \rightarrow \text{Tr} \rightarrow \text{Tr})$
- **tip_injective**: $\forall a \ b : A, \text{lit } a == \text{lit } b \rightarrow a == b$
- **fork_injective**: $\forall t \ u \ t' \ u' : \text{Tr}, \text{fork } t \ u == \text{fork } t' \ u' \rightarrow t == t' \wedge u == u'$
- **tip_not_fork**: $\forall (a : A) (t \ u : \text{Tr}), \text{tip } a \neq \text{fork } t \ u$
- **tree_induction**: $\forall P : \text{Tr} \rightarrow \text{Prop}, \text{Proper } (\text{equ} ==> \text{iff}) \ P \rightarrow (\forall a : A, P \ (\text{tip } a)) \rightarrow (\forall t \ u : \text{Tr}, P \ t \rightarrow P \ u \rightarrow P \ (\text{fork } t \ u)) \rightarrow \forall t : \text{Tr}, P \ t$

StackMachine (V C S) extends Monoid

ops: **binop** : $V \rightarrow V \rightarrow V$, **push_instr** : $V \rightarrow C$, **op_instr** : C , **exec** : $C \rightarrow S \rightarrow S$, **push** : $V \rightarrow S \rightarrow S$

- **values**: $\text{Equivalence } (@\text{equ } V \ \text{equ})$
- **stacks**: $\text{Equivalence } (@\text{equ } S \ \text{equ})$
- **binop_proper**: $\text{Proper } (\text{equ} ==> \text{equ} ==> \text{equ}) \ \text{binop}$
- **exec_proper**: $\text{Proper } (\text{equ} ==> \text{equ} ==> \text{equ}) \ \text{exec}$
- **push_proper**: $\text{Proper } (\text{equ} ==> \text{equ} ==> \text{equ}) \ \text{push}$
- **exec_empty**: $\forall s : S, \text{exec } 0 \ s == s$
- **exec_antiadditive**: $\forall x \ y : C, \text{exec } (x + y) == (\lambda s : S \Rightarrow \text{exec } y \ (\text{exec } x \ s))$
- **exec_push**: $\forall (v : V) (s : S), \text{exec } (\text{push_instr } v) \ s == \text{push } v \ s$
- **exec_op**: $\forall (a \ b : V) (s : S), \text{exec_op_instr } (\text{push } b \ (\text{push } a \ s)) == \text{push } (\text{binop } a \ b) \ s$

Compiler (V E C S) extends StackMachine, Tree

ops: **tip** : $V \rightarrow E$, **fork** : $E \rightarrow E \rightarrow E$, **binop** : $V \rightarrow V \rightarrow V$, **push_instr** : $V \rightarrow C$, **op_instr** : C , **exec** : $C \rightarrow S \rightarrow S$, **push** : $V \rightarrow S \rightarrow S$, **evaluate** : $E \rightarrow V$, **compile** : $E \rightarrow C$

- **evaluate_proper**: $\text{Proper } (\text{equ} ==> \text{equ}) \ \text{evaluate}$
- **compile_proper**: $\text{Proper } (\text{equ} ==> \text{equ}) \ \text{compile}$
- **evaluate_lit**: $\forall v : V, \text{evaluate } (\text{lit } v) == v$

- `evaluate_node`: $\forall e f : E, \text{evaluate}(\text{node } e f) == \text{binop}(\text{evaluate } e)(\text{evaluate } f)$
- `compile_lit`: $\forall v : V, \text{compile}(\text{lit } v) == \text{push_instr } v$
- `compile_node`: $\forall e f : E, \text{compile}(\text{node } e f) == \text{compile } e + (\text{compile } f + \text{op_instr})$

Dictionary (K V D)

ops: `empty` : $D, \text{insert} : K \rightarrow V \rightarrow D \rightarrow D, \text{find} : D \rightarrow K \rightarrow \text{option } V$

- `entries`: `Equivalence` (@`equ D equ`)
- `insert_proper`: `Proper` (`equ ==> equ ==> equ ==> equ`) (`insert : K → V → D → D`)
- `find_congruent`: $\forall (d : D) (k k' : K), k == k' \rightarrow \text{find } d k = \text{find } d k'$
- `find_empty`: $\forall k : K, \text{find}(\text{empty} : D) k = \text{None}$
- `find_insert_here`: $\forall (k k' : K) (v : V) (d : D), k == k' \rightarrow \text{exists } v' : V, \text{find}(\text{insert } k v d) k' = \text{Some } v' \wedge v' == v$
- `find_insert_there`: $\forall (k k' : K) (v : V) (d : D), k \neq k' \rightarrow \text{find}(\text{insert } k v d) k' = \text{find } d k'$

Lexicon (C N)

ops: `space` : $C \rightarrow \text{Prop}, \text{digit_value} : C \rightarrow \mathbb{N}, \text{backslash dot_c lparen rparen eqsign semi colon dash gt} : C$

- `characters`: `Equivalence` (@`equ C equ`)
- `space_proper`: `Proper` (`equ ==> iff`) `space`
- `alpha_proper`: `Proper` (`equ ==> iff`) `alpha`
- `digit_proper`: `Proper` (`equ ==> iff`) `digit`
- `digit_value_proper`: `Proper` (`equ ==> equ`) `digit_value`
- `space_dec`: $\forall c : C, \text{Decidable}(\text{space } c)$
- `alpha_dec`: $\forall c : C, \text{Decidable}(\text{alpha } c)$
- `digit_dec`: $\forall c : C, \text{Decidable}(\text{digit } c)$

Streaming (C S)

ops: `next` : $S \rightarrow \text{option } (C * S)$

- `next_decides_step`: $\forall (s : S) (c : C) (s' : S), \text{step } s c s' \leftrightarrow \text{exists } c0 : C, \text{next } s = \text{Some } (c0, s') \wedge c0 == c$

List (O K) over `Bicartesian 0` extends `InitialAlgebra`

ops: `build` : `shape den x L ~> L, cata` : $\forall x0 : 0, \text{shape den } x x0 \rightarrow x0 \rightarrow L \rightarrow x0, \text{den} : K \rightarrow 0, x : K, L : 0$

Either (O K) over `Bicartesian 0` extends `InitialAlgebra`

ops: `build` : `shape den a b E ~> E, cata` : $\forall x : 0, \text{shape den } a b x \rightarrow x \rightarrow E \rightarrow x, \text{den} : K \rightarrow 0, a b : K, E : 0$

Pair (O K) over `Bicartesian 0` extends `InitialAlgebra`

ops: `build` : `shape den a b P ~> P, cata` : $\forall x : 0, \text{shape den } a b x \rightarrow x \rightarrow P \rightarrow x, \text{den} : K \rightarrow 0, a b : K, P : 0$

NaturalNumbers (O) over `Bicartesian 0` extends `InitialAlgebra`

ops: `build` : `shape 0 N ~> N, cata` : $\forall x : 0, \text{shape } 0 x \rightarrow x \rightarrow N \rightarrow x, N : 0$

Tree (O K) over `Bicartesian 0` extends `InitialAlgebra`

ops: `build` : `shape den x T ~> T, cata` : $\forall x0 : 0, \text{shape den } x x0 \rightarrow x0 \rightarrow T \rightarrow x0, \text{den} : K \rightarrow 0, x : K, T : 0$

LambdaTerm (O K) over `Bicartesian 0` extends `InitialAlgebra`

ops: `build` : `shape den n T ~> T, cata` : $\forall x : 0, \text{shape den } n x \rightarrow x \rightarrow T \rightarrow x, \text{den} : K \rightarrow 0, n : K, T : 0$